sham.h
44 lines
H

server.c
445 lines
C

client.c
445 lines
C

V my chat mode is not working any issues??

Looking at your chat mode implementation, I can identify several critical issues that would prevent it from working correctly:

Major Issues:

1. Incorrect ACK Flag Check

```c
// In both client.c and server.c
if (ntohs(ACK) & ACK)  // This is WRONG!
```

Problem: You're checking `ntohs(ACK)` instead of `ntohs(response.a.flags)`. Since `ACK = 0x2`, this becomes `ntohs(0x2) & 0x2` which is always true.

Fix:

```c
if ((ntohs(response.a.flags) & ACK) == ACK)
```

2. Wrong Data Source in Packet Processing

```c
// In the "actual data" section
strcpy(data_recieved[0].data, data_recv.data);
data_recieved[0].len = ntohl(response.len);  // Should be data_recv.len
data_recieved[0].end = ntohl(response.a.seq_num) + data_recieved[0].len;  // Should be
```

Fix:

```c
strcpy(data_recieved[0].data, data_recv.data);
data_recieved[0].len = ntohl(data_recv.len);
data_recieved[0].end = ntohl(data_recv.a.seq_num) + data_recieved[0].len;
```

3. Inverted Quit Logic

```c
if (strcmp("/quit\n", buff))  // This returns 0 when strings match!
{
    quit = 1;
    one_done = 1;
    break;
}
```

Problem: `strcmp` returns 0 when strings are ↓ so this quits on any input EXCEPT "/

Problem: `strcmp` returns 0 when strings are equal, so this quits on any input EXCEPT "/quit".

Fix:

```c
if (strcmp("/quit\n", buff) == 0)
{
    quit = 1;
    one_done = 1;
    break;
}
```

4. Missing Flag Initialization in ACK Packets

Your ACK packets in the data reception logic don't initialize all fields:

```c
pack_str sending;
sending.a.ack_num = htonl(recieved);
sending.a.flags = htons(ACK);
// Missing: memset or initialization of other fields
```

Fix:

```c
pack_str sending;
```

```
// Missing: memset or initialization of other fields
```

Fix:

```c
pack_str sending;
memset(&sending, 0, sizeof(sending));
sending.a.ack_num = htonl(recieved);
sending.a.flags = htons(ACK);
```

5. Logic Flow Issues

The nested while loops and flag logic is overly complex and prone to deadlocks. The `select()` is called once but then you enter multiple nested loops that may never return to check for new socket activity.

Recommended Approach:

Consider simplifying the chat mode logic:

Simplified Chat Mode Logic
Code

The main issues causing your chat mode to fail are the incorrect flag checking and the inverted quit logic. Fix those first, and then consider simplifying the overall structure to avoid the complex nested loops that can cause deadlocks.

nd prone to deadlocks. The
ted loops that may never return to

correct flag checking and the
plifying the overall structure to avoid

s the error and gice me the code of

ors that need fixing:

```c
// Simplified chat mode structure
if (chat_mode) {
    not_ACKed data_received[10];
    not_ACKed data_sent[10];
    memset(data_received, 0, sizeof(data_received));
    memset(data_sent, 0, sizeof(data_sent));

    int sequence_num = 1;
    int expected_seq = 1;

    while (1) {
        fd_set readfds;
        FD_ZERO(&readfds);
        FD_SET(0, &readfds);          // stdin
        FD_SET(socketfd, &readfds);   // socket

        struct timeval timeout = {0, 100000}; // 100ms
        int activity = select(socketfd + 1, &readfds, NULL,

        // Handle socket input (incoming messages or ACKs)
        if (activity > 0 && FD_ISSET(socketfd, &readfds)) {
            pack_str response;
            socklen_t len = sizeof(client);
            int rec = recvfrom(socketfd, &response, sizeof(r
                               (struct sockaddr *)&client, &le

            if (rec > 0) {
                // Check if it's an ACK for our sent data
```

nd prone to deadlocks. The
ted loops that may never return to

correct flag checking and the
plifying the overall structure to avoid

s the error and gice me the code of

ors that need fixing:

```
pack_str response;
socklen_t len = sizeof(client);
int rec = recvfrom(socketfd, &response, sizeof(r
                   (struct sockaddr *)&client, &le

if (rec > 0) {
    // Check if it's an ACK for our sent data
    if ((ntohs(response.a.flags) & ACK) == ACK)
        int ack_num = ntohl(response.a.ack_num);
        // Remove acknowledged packets from send
        for (int i = 0; i < 10; i++) {
            if (data_sent[i].len > 0 && data_sen
                memset(&data_sent[i], 0, sizeof(
            }
        }
    }
    // Handle incoming data
    else if (response.len > 0) {
        // Process incoming message
        uint32_t recv_seq = ntohl(response.a.seq
        int data_len = ntohl(response.len);

        if (recv_seq == expected_seq) {
            // In-order packet
            printf("Received: %s", response.data
            expected_seq += data_len;

            // Send ACK
            pack_str ack_pkt;
```

nd prone to deadlocks. The
ted loops that may never return to

correct flag checking and the
plifying the overall structure to avoid

s the error and gice me the code of

ors that need fixing:

```c
// Handle user input
if (activity > 0 && FD_ISSET(0, &readfds)) {
    char buffer[1024];
    if (fgets(buffer, sizeof(buffer), stdin)) {
        if (strcmp(buffer, "/quit\n") == 0) {
            break;
        }

        // Find free slot in send buffer
        int free_slot = -1;
        for (int i = 0; i < 10; i++) {
            if (data_sent[i].len == 0) {
                free_slot = i;
                break;
            }
        }

        if (free_slot >= 0) {
            // Prepare and send packet
            pack_str pkt;
            memset(&pkt, 0, sizeof(pkt));
            pkt.a.seq_num = htonl(sequence_num);
            strcpy(pkt.data, buffer);
            pkt.len = htonl(strlen(buffer));

            // Store in send buffer for retransmissi
            data_sent[free_slot].len = strlen(buffer
            strcpy(data_sent[free_slot].data, buffer
```

nd prone to deadlocks. The
ted loops that may never return to

correct flag checking and the
plifying the overall structure to avoid

s the error and gice me the code of

ors that need fixing:

```
            data_sent[free_slot].end = sequence_num
            gettimeofday(&data_sent[free_slot].time,

            // Send packet
            int to_send = sizeof(pkt.a) + sizeof(pkt
            sendto(socketfd, &pkt, to_send, 0, (stru
                   sizeof(client));

            sequence_num += strlen(buffer);
        }
    }
}

// Handle retransmissions
struct timeval now;
gettimeofday(&now, NULL);
for (int i = 0; i < 10; i++) {
    if (data_sent[i].len > 0 && elapsed_ms(data_sent
        // Retransmit packet
        pack_str pkt;
        memset(&pkt, 0, sizeof(pkt));
        pkt.a.seq_num = htonl(data_sent[i].end - dat
        strcpy(pkt.data, data_sent[i].data);
        pkt.len = htonl(data_sent[i].len);

        int to_send = sizeof(pkt.a) + sizeof(pkt.len
        sendto(socketfd, &pkt, to_send, 0, (struct s
               sizeof(client));
```

Key Errors Fixed:

1. **ACK Flag Check:** Changed `ntohs(ACK) & ACK` to `(ntohs(response.a.flags) & ACK) == ACK`
2. **Quit Logic:** Changed `strcmp("/quit\n", buff)` to `strcmp("/quit\n", buff) == 0`
3. **Data Source:** Fixed using `data_recv` instead of `response` for packet data and sequence numbers
4. **Socklen_t:** Fixed `sizeof(&client)` to `sizeof(client)` and `sizeof(&server_addr)` to `sizeof(server_addr)`
5. **Missing memset:** Added `memset()` calls for packet initialization to prevent garbage data
6. **Missing field reset:** Added `data_recieved[i].len = 0` in the cleanup loop
7. **Printf format:** Fixed the printf statement in server to include newline and proper format

These are the exact errors causing your chat mode to malfunction. The logic and variable structure remain the same as you requested.

```
                flag2 = 0;
            }
        }
        if (flag1 & flag2)
        {
            pack_str pc;
            pc.a.flags = htons(FIN);
            sendto(socketfd, &pc, sizeof(pc), 0, (struct sockaddr
*)&server_addr, len);
            four_way_hand_shake_sender(socketfd, &server_addr);
            done = 1;
        }
    }
    printf("File Transfer Succesfull\n");
  }
  close(socketfd);
  return 0;
}
```

in chat mode when i enter /quit its not succesfully doing the four way hand shake

c

Copy code

```c
// Replace both functions with the following

#include <sys/time.h>

void four_way_hand_shake_reciever(int socketfd, struct sockaddr_in *sender)
{
    printf("Receiver: received FIN\n");
    pack_str pkt;
    socklen_t len = sizeof(*sender);

    // Send ACK for the FIN we got
    memset(&pkt, 0, sizeof(pkt));
    pkt.a.flags = htons(ACK);
    if (sendto(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, len) < 0)
        perror("send ACK (receiver)");

    printf("Receiver: Sent ACK\n");

    // Send our FIN
    memset(&pkt, 0, sizeof(pkt));
    pkt.a.flags = htons(FIN);
    if (sendto(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, len) < 0)
        perror("send FIN (receiver)");
```

```c
printf("Receiver: Sent ACK\n");

// Send our FIN
memset(&pkt, 0, sizeof(pkt));
pkt.a.flags = htons(FIN);
if (sendto(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, len) < 0)
    perror("send FIN (receiver)");

printf("Receiver: Sent FIN\n");

// Wait for final ACK (with a safety timeout & retries)
struct timeval start, now;
gettimeofday(&start, NULL);
while (1)
{
    memset(&pkt, 0, sizeof(pkt));
    int r = recvfrom(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, &len);
    if (r < 0)
    {
        // nonfatal; keep waiting
        perror("recvfrom (receiver)");
        continue;
    }

    uint16_t flags = ntohs(pkt.a.flags);
    if ((flags & ACK) == ACK)
    {
```

```
        }

        uint16_t flags = ntohs(pkt.a.flags);
        if ((flags & ACK) == ACK)
        {
            printf("Receiver: Received Final ACK\n");
            break;
        }

        // optional safety: if we wait too long, resend our FIN once
        gettimeofday(&now, NULL);
        long waited_ms = (now.tv_sec - start.tv_sec) * 1000 + (now.tv_usec - start.tv_usec
        if (waited_ms > 2000) {
            // resend FIN once
            memset(&pkt, 0, sizeof(pkt));
            pkt.a.flags = htons(FIN);
            if (sendto(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, len) < 0
                perror("resend FIN (receiver)");
            gettimeofday(&start, NULL); // reset timer
        }
    }
}

void four_way_hand_shake_sender(int socketfd, struct sockaddr_in *sender)
{
    pack_str pkt;
    socklen_t len = sizeof(*sender);
```

```
        }
    }
}

void four_way_hand_shake_sender(int socketfd, struct sockaddr_in *sender)
{
    pack_str pkt;
    socklen_t len = sizeof(*sender);

    // Send FIN and keep retransmitting until we receive ACK for it
    memset(&pkt, 0, sizeof(pkt));
    pkt.a.flags = htons(FIN);

    struct timeval first_send;
    gettimeofday(&first_send, NULL);
    if (sendto(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, len) < 0)
        perror("send FIN (sender)");
    printf("Initiator: Sent FIN\n");

    struct timeval last_tx = first_send;
    while (1)
    {
        // Try to receive something
        pack_str rcv;
        socklen_t rlen = len;
        int r = recvfrom(socketfd, &rcv, sizeof(rcv), 0, (struct sockaddr *)sender, &rlen);
        if (r < 0)
```

```
pack_str rcv;
socklen_t rlen = len;
int r = recvfrom(socketfd, &rcv, sizeof(rcv), 0, (struct sockaddr *)sender, &rlen)
if (r < 0)
{
    perror("recvfrom (sender)");
    // check retransmit timer below
} else {
    uint16_t flags = ntohs(rcv.a.flags);

    // If it's ACK for our FIN -> proceed to wait for peer FIN
    if ((flags & ACK) == ACK)
    {
        printf("Initiator: Received ACK for our FIN\n");
        break;
    }

    // If peer already sent FIN -> we can send final ACK and exit
    if ((flags & FIN) == FIN)
    {
        printf("Initiator: Received FIN (peer) before ACK; send final ACK and exit
        // send final ACK
        pack_str ackp;
        memset(&ackp, 0, sizeof(ackp));
        ackp.a.flags = htons(ACK);
        if (sendto(socketfd, &ackp, sizeof(ackp), 0, (struct sockaddr *)sender, le
            perror("send final ACK (sender)");
```

```
            memset(&ackp, 0, sizeof(ackp));
            ackp.a.flags = htons(ACK);
            if (sendto(socketfd, &ackp, sizeof(ackp), 0, (struct sockaddr *)sender, le
                perror("send final ACK (sender)");
            return;
        }
    }

    // Retransmit FIN every 500ms until we get ACK
    struct timeval now;
    gettimeofday(&now, NULL);
    long elapsed = (now.tv_sec - last_tx.tv_sec) * 1000 + (now.tv_usec - last_tx.tv_us
    if (elapsed > 500)
    {
        if (sendto(socketfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)sender, len) < 0
            perror("re-send FIN (sender)");
        last_tx = now;
        printf(".");
        fflush(stdout);
    }
}

// Now we got ACK for our FIN. Wait for peer FIN (with a small timeout and retransmit
printf("\nInitiator: Waiting for peer FIN...\n");
struct timeval wait_start;
gettimeofday(&wait_start, NULL);
```

```
    }
}

// Now we got ACK for our FIN. Wait for peer FIN (with a small timeout and retransmit
printf("\nInitiator: Waiting for peer FIN...\n");
struct timeval wait_start;
gettimeofday(&wait_start, NULL);

while (1)
{
    pack_str rcv;
    socklen_t rlen = len;
    int r = recvfrom(socketfd, &rcv, sizeof(rcv), 0, (struct sockaddr *)sender, &rlen)
    if (r < 0)
    {
        perror("recvfrom (sender waiting FIN)");
    } else {
        uint16_t flags = ntohs(rcv.a.flags);
        if ((flags & FIN) == FIN)
        {
            printf("Initiator: Received FIN from peer\n");
            // Send final ACK
            pack_str ackp;
            memset(&ackp, 0, sizeof(ackp));
            ackp.a.flags = htons(ACK);
            if (sendto(socketfd, &ackp, sizeof(ackp), 0, (struct sockaddr *)sender, le
                perror("send final ACK (sender)");
```

```
                memset(&ackp, 0, sizeof(ackp));
                ackp.a.flags = htons(ACK);
                if (sendto(socketfd, &ackp, sizeof(ackp), 0, (struct sockaddr *)sender, le
                    perror("send final ACK (sender)");
                printf("Initiator: Sent Final ACK\n");
                break;
            }
        }

        // safety: if peer's FIN doesn't arrive in, say, 3s, resend final ACK (in case pee
        struct timeval now;
        gettimeofday(&now, NULL);
        long waited = (now.tv_sec - wait_start.tv_sec) * 1000 + (now.tv_usec - wait_start.
        if (waited > 3000) {
            // send a final ACK anyway (defensive)
            pack_str ackp;
            memset(&ackp, 0, sizeof(ackp));
            ackp.a.flags = htons(ACK);
            if (sendto(socketfd, &ackp, sizeof(ackp), 0, (struct sockaddr *)sender, len) <
                perror("resend final ACK (sender)");
            gettimeofday(&wait_start, NULL);
        }
    }
}
```

Notes & recommended small changes

okay lets start cfs

Show thinking

Alright, let's dive into the **Completely Fair Scheduler (CFS)**. This is the most complex part, but we'll tackle it step-by-step.

The goal of CFS is to give each process a fair share of the CPU. It does this by tracking a "virtual runtime" (`vruntime`) for each process. The scheduler always picks the runnable process with the **lowest** `vruntime` .

Here are the first steps, focusing on adding the necessary process attributes.

Step 1: Add CFS Fields to `struct proc`

We need to add several new fields to the process structure to track CFS-specific data like priority (`nice`), virtual runtime (`vruntime`), and time slice information.

- **File:** `kernel/proc.h`
- **Action:** Add the new fields to `struct proc` .

(`nice`), virtual runtime (`vruntime`), and time slice information.

- **File:** `kernel/proc.h`
- **Action:** Add the new fields to `struct proc` .

C

```c
// In struct proc in kernel/proc.h
struct proc {
  // ... existing fields like pid, state, ctime ...

  // Add these fields for CFS
  int nice;                     // Priority value from -20 to 19
  uint vruntime;                 // Virtual runtime
  int ticks_in_slice;           // Ticks consumed in the current time slice
  int time_slice;               // The duration of this process's time slice

  // ... rest of the struct
};
```

Step 2: Initialize New Fields in `allocproc`

When a new process is created, we must initialize its CFS values to a sensible default.

- **File:** `kernel/proc.c`
- **Action:** In the `allocproc()` function, add initialization for the new fields. A `nice` value of 0 is the standard default priority.

C

```c
// In allocproc() in kernel/proc.c, inside the 'found:' block
found:
  p->pid = allocpid();
  p->state = USED;
```

Ask Gemini

```c
  p->ctime = counter++; // Your FCFS counter

  // Add this block to initialize CFS fields
  p->nice = 0;           // Default priority
  p->vruntime = 0;       // Start with zero virtual runtime
  p->ticks_in_slice = 0;
  p->time_slice = 0;

  // ... rest of the function ...
```

Step 3: Implement Priority-to-Weight Conversion

CFS uses a `weight` value derived from the `nice` value to adjust `vruntime` updates. A higher weight (lower `nice` value) means `vruntime` grows more slowly, causing the process to be picked more often. We'll create a lookup table for this conversion as it's much more efficient than calculating `1024 / (1.25 ^ nice)` every time.

- **File:** `kernel/proc.c`
- **Action:** Add the `nice_to_weight` lookup table and a helper function at the top of the file.

```c
```

calculating `1024 / (1.25 ^ nice)` every time.

- **File:** `kernel/proc.c`
- **Action:** Add the `nice_to_weight` lookup table and a helper function at the top of the file.

c

```c
// At the top of kernel/proc.c, near the other includes/declarations

// nice values range from -20 to 19.
// This table maps nice values (index + 20) to CFS weights.
static const int nice_to_weight[40] = {
 /* -20 */ 88761, 71755, 56483, 46273, 36291,
 /* -15 */ 29154, 23254, 18705, 14949, 11916,
 /* -10 */  9548,  7620,  6100,  4904,  3906,
 /* -5 */  3121,  2501,  1991,  1586,  1277,
 /* 0 */  1024,   820,   655,   526,   423,
 /* 5 */   335,   272,   215,   172,   137,
 /* 10 */   110,    87,    70,    56,    45,
 /* 15 */    36,    29,    23,    18,    15,
};

// Helper function to get the weight for a given nice value
static int get_weight(int nice) {
  return nice_to_weight[nice + 20];
}
```

k Gemini

- **File:** `kernel/proc.c`
- **Action:** Add the new `#elif` block to your `scheduler()` function.

c

```c
// In scheduler() in kernel/proc.c
void
scheduler(void)
{
  // ... (existing variable declarations) ...
  for(;;){
    intr_on();

#if defined(FCFS)
    // ... your FCFS code ...

#elif defined(CFS)
    // CFS SCHEDULER
    struct proc *min_vruntime_proc = 0;
    int runnable_procs = 0;

    // First, count runnable processes
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        runnable_procs++;
      }
    }
```

```c
// First, count runnable processes
for(p = proc; p < &proc[NPROC]; p++) {
  if(p->state == RUNNABLE) {
    runnable_procs++;
  }
}

if(runnable_procs > 0) {
  // Find the process with the minimum vruntime
  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->state == RUNNABLE) {
      if(min_vruntime_proc == 0 || p->vruntime < min_vruntime_proc->vruntime) {
        if(min_vruntime_proc)
          release(&min_vruntime_proc->lock);
        min_vruntime_proc = p;
        continue;
      }
    }
    release(&p->lock);
  }
}

if(min_vruntime_proc) {
  // Calculate the time slice for the chosen process
  int target_latency = 48; // in ticks
  int min_timeslice = 3;   // in ticks
```

```c
        }
      }
      release(&p->lock);
    }
  }

  if(min_vruntime_proc) {
    // Calculate the time slice for the chosen process
    int target_latency = 48; // in ticks
    int min_timeslice = 3;   // in ticks

    min_vruntime_proc->time_slice = target_latency / runnable_procs;
    if (min_vruntime_proc->time_slice < min_timeslice) {
      min_vruntime_proc->time_slice = min_timeslice;
    }
    min_vruntime_proc->ticks_in_slice = 0; // Reset slice counter

    // Run the chosen process
    p = min_vruntime_proc;
    p->state = RUNNING;
    c->proc = p;
    swtch(&c->context, &p->context);
    c->proc = 0;
    release(&p->lock);
  }

#else
```

c

Step 5: Modify `usertrap()` for vruntime and Preemption

This is the most critical part of CFS. On every timer interrupt, we need to update the running process's `vruntime` and check if it has used up its time slice.

- **File:** `kernel/trap.c`
- **Action:** Modify the timer interrupt handler in `usertrap()`.

c

```c
// In usertrap() in kernel/trap.c

// give up the CPU if this is a timer interrupt.
if(which_dev == 2) {
  #if defined(FCFS)
    // FCFS is non-preemptive, do nothing.
  #elif defined(CFS)
    // CFS vruntime update and preemption logic
    struct proc *p = myproc();
    if(p && p->state == RUNNING) {
      int weight0 = get_weight(0); // Weight of a default-priority process
      int weight_p = get_weight(p->nice);
```

c

```c
if(which_dev == 2) {
  #if defined(FCFS)
    // FCFS is non-preemptive, do nothing.
  #elif defined(CFS)
    // CFS vruntime update and preemption logic
    struct proc *p = myproc();
    if(p && p->state == RUNNING) {
      int weight0 = get_weight(0); // Weight of a default-priority process
      int weight_p = get_weight(p->nice);

      // Update vruntime. It increases by a factor of (weight0 / weight_p).
      // A high-priority process (high weight) will have its vruntime increase slo
      p->vruntime += (weight0 * 100) / weight_p; // Scaled by 100 to avoid floats

      p->ticks_in_slice++;
      if(p->ticks_in_slice >= p->time_slice) {
        yield(); // Time slice is up, yield the CPU.
      }
    }
  #else
    // Default round-robin scheduler
    yield();
  #endif
}
```

- **Note:** To make this work, you must make the `get_weight()` function available to `trap.c`.

- **Note:** To make this work, you must make the `get_weight()` function available to `trap.c`.
    1. In `kernel/proc.c`, remove the `static` keyword from `get_weight` to make it a global function: `int get_weight(int nice) { ... }`
    2. In `kernel/defs.h`, add its prototype: `int get_weight(int);`

Final Step: Build and Test

You are now ready to compile your CFS scheduler.

1. **Build:**

```
Bash
make clean && make qemu SCHEDULER=CFS
```

2. **Run your test program:**

```
Bash
$ cpubound
```

With CFS, the output should be interleaved like the default scheduler (e.g., `ABCABC...`), but the

give me cpu bound code

Show thinking

Of course. Here is a C code snippet designed for testing schedulers in an environment like xv6. It creates purely CPU-bound processes that perform a long calculation without making any I/O system calls until the very end.

This ensures that the scheduler's behavior is what you're testing, not the side effects of I/O.

`cpubound.c` - A Purely CPU-Bound Test

c

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

// A purely CPU-bound task that does not perform I/O in its main loop.
void cpu_task(char id) {
```

sk Gemini

c

```c
#include "kernel/stat.h"
#include "user/user.h"

// A purely CPU-bound task that does not perform I/O in its main loop.
void cpu_task(char id) {
    // Perform a meaningless but time-consuming calculation.
    long result = 0;
    for (long i = 0; i < 25000000; i++) {
        result += (id * i) % 1234;
    }
    // Print ONLY once at the very end to signal completion.
    printf("Process %c finished.\n", id);
}

int main(void) {
    printf("Starting purely CPU-bound test...\n");

    if (fork() == 0) { // Child A
        cpu_task('A');
        exit(0);
    }

    if (fork() == 0) { // Child B
        cpu_task('B');
        exit(0);
    }
```

```c
    if (fork() == 0) { // Child B
        cpu_task('B');
        exit(0);
    }

    if (fork() == 0) { // Child C
        cpu_task('C');
        exit(0);
    }

    // Parent waits for all three children to finish
    wait(0);
    wait(0);
    wait(0);

    printf("Test finished.\n");
    exit(0);
}
```

expected output is it constantly same output??

sk Gemini

expected output is it constantly same output??

Show thinking

No, the expected output is **different** for each scheduler. The whole point of the test is to see how their different logic produces different behavior.

Here's a summary of what you should expect from the purely CPU-bound test program.

1. Default (Round Robin)

The default scheduler is **preemptive**. It gives each process a small turn and then forcibly switches to the next, regardless of whether the first is finished.

Expected Output

The output will be **interleaved and unpredictable**. Because the scheduler constantly interrupts the processes, the one that finishes first is mostly random.

sk Gemini

1. Default (Round Robin)

The default scheduler is **preemptive**. It gives each process a small turn and then forcibly switches to the next, regardless of whether the first is finished.

Expected Output

The output will be **interleaved and unpredictable**. Because the scheduler constantly interrupts the processes, the one that finishes first is mostly random.

```
Starting purely CPU-bound test...
Process B finished.
Process C finished.
Process A finished.
Test finished.
```

(The order A, B, C will likely be different each time you run it.)

---

2. FCFS (First Come, First Serve)

Your FCFS scheduler is **non-preemptive**. It picks the first process created (`A`) and lets it run to completion without any interruptions.

```
Process A finished.
Test finished.
```

(The order A, B, C will likely be different each time you run it.)

2. FCFS (First Come, First Serve)

Your FCFS scheduler is **non-preemptive**. It picks the first process created (`A`) and lets it run to completion without any interruptions.

Expected Output

The output will be **perfectly sequential and constant**. It will always be A, then B, then C.

```
Starting purely CPU-bound test...
Process A finished.
Process B finished.
Process C finished.
Test finished.
```

(You will notice a distinct pause between each line.)

3. CFS (Completely Fair Scheduler)

```
Test finished.
```

(You will notice a distinct pause between each line.)

3. CFS (Completely Fair Scheduler)

CFS is **preemptive and fair**. Like Round Robin, it switches between processes. However, it tries to give each process an equal "virtual" amount of CPU time.

Expected Output

The output will be **interleaved but often more consistent than Round Robin**. Since all processes have the same priority (`nice=0`), CFS will try to divide the CPU time evenly between them. They will likely finish around the same time, but in an unpredictable order.

```
Starting purely CPU-bound test...
Process C finished.
Process A finished.
Process B finished.
Test finished.
```

*(The key is that it will **not** be the strict `A -> B -> C` sequence of FCFS.)*